Modeling the Autoscaling Operations in Cloud with Time Series Data

Mehran N. A. H. Khan, Yan Liu, Hanieh Alipour, Samneet Singh
Electrical and Computer Engineering Department
Concordia University, Montreal, QC, Canada
Yan.liu@concordia.ca ;{meh_khan; h_alipour ;sam_sing}@encs.concordia.ca

***Abstract*—Autoscaling involves complex cloud operations that automate the provisioning and de-provisioning of cloud resources to support continuous development of customer services. Autoscaling depends on a number of decisions derived by aggregating metrics at the infrastructure and the platform level. In this paper, we review existing autoscaling techniques deployed in leading cloud providers. We identify core features and entities of the autoscaling operations as variables. We model these variables that quantify the interactions between these entities and incorporate workload time series data to calibrate the model. Hence the model allows proactive analysis of workload patterns and estimation of the responsiveness of the autoscaling operations. We demonstrate the use of this model with Google cluster trace data.**

Keywords—Autoscaling, cloud operation, modeling, time series

I. INTRODUCTION

In cloud computing, the enormous service demands are split and distributed over a network according to the clients' proximity to a network region and tasks-types to provide the acceptable level of services [1]. Autoscaling enables the option to scale in or scale out the cloud resources based on predefined policies, status checks and schedules for the cloud service users [2]. The precision of autoscaling operations depends on a number of factors that are derived by aggregating metrics such as workloads on virtual machines, network traffic, queue length at the platform level, and so on. Eventually, the result of aggregation drives the decisions to provision or de-provision the amount of resource for a particular service or application running on the cloud. The efficiency of autoscaling operations has a direct impact on the quality and cost of a cloud service from the customer point of view. An efficient autoscaling needs to meet the following requirements:

- Precise analysis of monitoring data in time series. Monitoring tools can produce metrics at various granularity levels, in terms of time resolution and frequency. Hence, a rule-based autoscaling control process often depends on the aggregated metrics consistent with the user-defined rules.
- Awareness of the states of the system as well as the related entities participating in the autoscaling process. For example, the responsiveness of the entity performing scaling-in and scaling-out cloud operations affects the end-to-end delay of the overall autoscaling process;
- Capturing the usage patterns in a timely manner helps to reduce the risks of missing workloads with spikes that need prompt reaction to meet the resource demands [2];
- Cost effectiveness. The autoscaling process normally incurs no direct cost from the cloud provider. However, the monitoring tools or services can be charged in the unit of messages produced or the number of metrics monitored. Moreover, the storage usage of the monitoring data is billed.

Current techniques of autoscaling at the Infrastructure-as-a-Service (IaaS) level are mainly categorized into five aspects [1]; namely, (1) static and threshold-based polices, (2) reinforcement learning, (3) queueing theory, (4) control theory and (5) time series analysis. In particular, time series analysis is often applied to the autoscaling problem as prediction of a future resource usage. A time series is a sequence of data points, measured typically at successive time instants spaced at uniform time intervals. Based on this predicted value using time series analysis, the decision can be made on the suitable scaling action to take.

Incorporating the time series data into autoscaling operations provides a fine-grained control of the autoscaling workflow. Existing autoscaling techniques normally take only a snapshot of values within specific time intervals according to static settings. However, the time series data introduce an extra level of uncertainty to the decision making process. For example, the reaction time and success rate of the elasticity engine spent as a response to a scaling-out or scaling-in action vary at runtime and even demonstrate a long tail behavior [3]. Therefore, modeling the intrinsic relations among the metrics critical to autoscaling operations and analyzing their effects over continuous time periods provide insights to explore a precise and cost effective control of the autoscaling process.

In this paper, we review existing autoscaling operations and techniques deployed in cloud providers. We identify the features and core entities from these operations as variables. We model the variables that quantify the interactions between these entities and incorporate time series analysis. Hence the model enables analysis of relation between the autoscaling operations and workload patterns as well as estimation of the responsiveness of the autoscaling operations. We demonstrate the use of this model using Google cluster trace data.

II. OVERVIEW OF CLOUD AUTOSCALING OPERATIONS

We present an overview of autoscaling operations of commercial cloud service providers. The purpose is to identify common features and entities from these operations.

978-1-5090-0092-0/15 $31.00 © 2015 IEEE
DOI 10.1109/SRDSW.2015.20

A. AWS Auto Scaling

Amazon EC2 [5] offers rule-based autoscaling as a Web service to automatically add or remove Amazon EC2 instances based on health checking and user-defined policies. Figure 1 illustrates the AWS autoscaling architecture that consists of five main parts:

- *The Auto Scaling Group:* Responsible for grouping the EC2 instances for scaling and managing the instances, based on the minimum and maximum number of running EC2 instances the group is allowed at any time.
- *Launch configuration:* Provides all the necessary information that is required to instantiate EC2 instances.
- *CloudWatch:* Monitors the load on VM instances based on defined metrics (such as CPU, memory utilization, or other system level loads).
- *Scaling Policy*: A set of policies for scaling in and out.

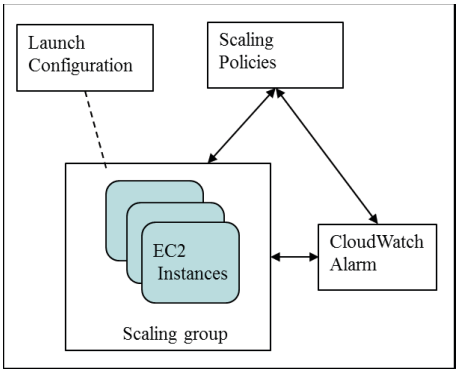

Figure 1 : AWS autoscaling architecture

The main idea of AWS Autoscaling is that instances with shared characteristics can be categorized into an Auto Scaling Group (ASG) so that a single launch configuration includes all necessary attributes to launch or terminate instances.

In addition, AWS's application management service called OpsWorks offers a flexible way for creating and managing stacks (groups of AWS resources such as EC2 instances and EBS volumes) and applications. It scales services and applications automatically based on time and load, and also uses dynamic configuration to orchestrate changes. OpsWorks contains three types of instances, namely 24/7, load-based and time-based. The 24/7 instances are started and stopped manually. In contrast, load-based instances and time-based instances are automatically started and stopped by AWS OpsWorks. Load-based scaling starts or stops instances in response to changes in loads. Time-based scaling starts or stops instances according to a specified schedule; i.e. instances are added and removed only at certain times. Load-based instances are suitable for handling fluctuated loads, whereas time-based instances are useful when there are predictable loads.

B. OpenStack Heat Autoscaling

In OpenStack, two components, namely *Heat* and *Ceilometer* work together to enable the autoscaling feature [6] as shown in Figure 2. *Heat* orchestrates complex deployments on top of virtual machines. Meanwhile, *Ceilometer*'s responsibility is to retrieve load information from the virtual machines. If a threshold is reached, the *Ceilometer* invokes the alarm action to notify the Heat API to scale in or out.

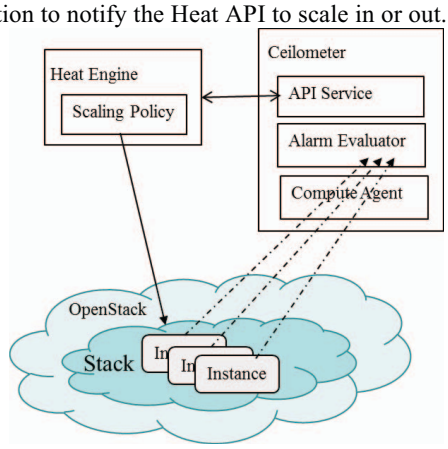

Figure 2: OpenStack auto-scaling architecture

C. Microsoft Autoscaling Application Block

The Autoscaling Application Block (AAB) [7] is part of the Microsoft Azure cloud platform that defines and monitors autoscaling behavior in Azure applications.

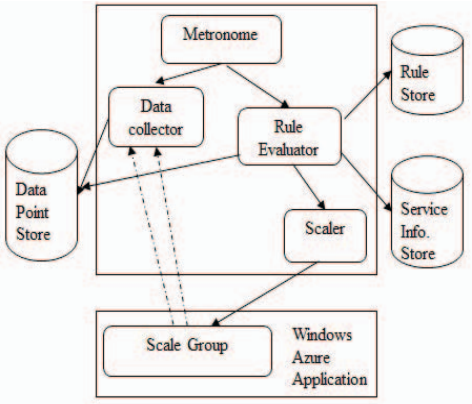

Figure 3 Azure Autoscaling Application Block

The architecture in Figure 3 shows the main components, as described below:

- The *Autoscaler* class is a façade for the AAB. This class initializes and then starts the autoscaling behaviour.
- The *Metronome* class runs activities on a regular schedule and it is responsible for launching all of the activities that the AAB performs. Each activity can have its own schedule.
- The *Sampler* class collects data points from the Azure environment and then saves the data points to the *Data Points Store*. A data point is the value of a specific metric (such as memory usage or CPU utilization) at specific time. This data is used for deciding actions in AAB.
- The *Service Information Store* stores the operational configuration, which is the service model of an Azure application. This service model includes all of the information about role names and storage account details that the block needs to be able to collect data points from

the target Azure application and to perform scaling operations.

- The *Rule Evaluator*. Whenever an unexpected load occurs, the *Rule Evaluator* communicates with the *Data Points Store* to analyze the collected data. Additionally, it communicates with the *Rule Store* to determine which autoscaling rule to apply.
- The *Rule Store* stores the lists of all autoscaling rules.
- The *Scaler* is responsible for adding and removing Azure instances.

D. Google Compute Engine Autoscaler

Google [8] offers the autoscaling function that relies on the combination of *Google Compute Engine*, *Google App Engine*, and *Cloud Datastore*. The *Google App Engine* monitors and orchestrates the autoscaling of the *Compute Engine* instances. This monitoring includes CPU, memory and other system load data. The autoscaling starts by reading a XML configuration file, which contains the rules for scaling the instances in or out. Figure 4 demonstrates the autoscaling architecture.

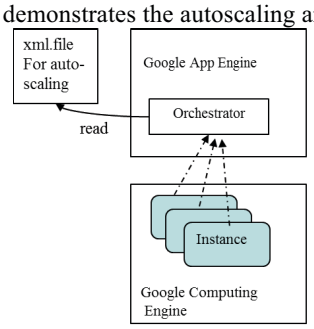

Figure 4: Google cloud autoscaling architecture

E. Summary

In a nutshell, the above autoscaling operations involve three main entities, namely, a health monitor that collects the metrics at the infrastructure or platform level; a user-defined launch configuration that captures the parameters necessary to create and terminate resources in the autoscaling mode; and a logic autoscaling group for resource management. In addition, a load balancing component (such as load balancer or queue) is attached to balance or level the workload on a service to all available computing nodes. All these entities are coordinated by an autoscaling process encapsulated by a controller that uses the status collected by the monitor and invokes rule-based scaling actions.

III. MODELING AUTOSCALING OPERATIONS

From the above reviews of the autoscaling operations, we observe these operations mainly adopt the rule-based approach by using a set of metrics collected at the infrastructure and the platform levels. We present a model that consists of parameters whose values calibrate the autoscaling workflow in time series data collected for these metrics.

A. Modeling Autoscaling States

We consider the states related to under-provision and over-provision respectively. The variables related to the under-provisioned states are: *thrUp*, *vUp* and *inUp*. The conditions that lead to under-provisioned state are as follows:

1. The load curve crosses the *thrUp* threshold. The condition is represented by ↑*thrUp*
2. The load curve remains over the *thrUp* threshold line for *vUp* amount of time. The condition is represented by ↑*vUp*
3. Time that passes since any alert is triggered is longer than *inUp*. The condition is represented by ↑*inUp*
4. Autoscaling action success (the Boolean value of the variable *autoscalingSuccess*): ↑*success*

Likewise, *thrDown*, *vDown*, *inDown* are the variables related to over-provisioned states. ↑*thrDown*, ↑*vDown* and ↑*inDown* are the corresponding conditions that lead to over-provisioned states. Since all aforementioned conditions are Booleans, the collection of them can be put into a condition combination table to list all valid conditions that trigger meaningful states transition.

Table 1 Condition combination of under-provisioned state

Condition				Verdict
↑thrUp	**↑vUp**	**↑inUp**	**↑Success**	
F	F	F	-	W/O Autoscaling
T	F	F	F	Under-provisioned less duration and no autoscaling action
T	T	F	F	Provision is Triggered but autoscaling fails
T	T	F	T	Autoscaling succeeds
T	T	T	F	Provision alert is triggered, but autoscaling process fails to respond
T	T	T	T	Provision alert is triggered, and autoscaling process succeeds

B. Autoscaling with Time Series Data

Based on the above analysis of conditions that triggers autoscaling transitions, we further present the workflow in List 1 that takes time series monitoring data as inputs for the autoscaling operations.

INPUTS:

- *X,* predicted workload in time series matrix;
- *thrUp,* upper threshold in time series matrix;
- *thrDown,* lower threshold in time series matrix;
- *vUp,* time that upper threshold has been across before requesting provisioning alert;
- *vDown,* time that lower threshold has been across before requesting deprovisioning alert;
- *inUp,* time required for a provisioning action to respond;
- *inDown,* time required for a deprovisioning action to respond;

- *autoscalingSuccess*, a binary flag controlled by the Elasticity Engine to signal provisioning action success//failure.

OUTPUT:

{overprovisioned ,underprovisioned } $\in A$, A is the set of possible alerts to be triggered

VARIABLES:

- $t, \mid t \in T$, T is the set of all possible time object variables (i.e. time object variable *index* | *index* $\in T$ acts as an index of the time series matrices);
- *index*, t_u , t_{inUp}, $t_{inDown} \in T$;
- $Z \mid Z \in T$, z is the time duration of one partition of the analysis;

$I \mid I \in T$, I is a single unit of observation variable. It represents the unit of data point in the time series.

```
while ¬IsNull(X(t+I))
    index ← 0
    if (X(t) > thrUp(t) and ¬AlertTriggeredIn(inUp)
    and ¬AlertTriggeredIn(inDown) )
        t_u ← t + vUp
        while ((t + index) < t_u and X(t + index)
        >thrUp(t) )
            index ← index + I
        end loop
        t ← t + index
        if (t ≥ t_u)
            Alert(underprovisioned)
            index ← 0
            t_inUp ← t + inUp
            autoscalingSuccess ← false
            while(t+I < t_inUp)
                t ← t + I
                Wait(I)
            end loop
            autoscalingSuccess ← QueryAutoscalingSuccess()
            Report(autoscalingSuccess)
             if (¬autoscalingSuccess)
                if (inUp > z)
                    Wait(Z – inUp)
                    t ← t + (Z – inUp)
                else
                    Wait(inUp – Z)
                    t ← t + (inUp – Z)
                end if
             end if
        index ← 0
        t_inUp ← 0
        t_u ← 0
        autoscalingSuccess ← false
      end if
    end if
    t ← t + I
end loop
```

List 1: Autoscaling workflow with monitoring data in time series

IV. THE USE OF THE MODEL

A. Workload Prediction

The predicted workload X in time series is the main data input, while other parameters are defined in the autoscaling configuration. The prediction is twofold. For a short term prediction, a smoothing method, such as the Exponential Smoothing takes previous pattern of the workload curve into account. For long term prediction, the states of the system is classified into a number of distinct, well defined states. Based on observation and statistical data, a Markov Chain Model is formulated from which a probability of the upcoming state of the system can be derived [1]. Based on these predictions, the system can adapt to the workload change before the workload actually occurs, which in turn, minimizes the delay to take provisioning/de-provisioning action in the cloud architecture. In this paper, we consider the Exponential Smoothing method for predicting the moving average of the upcoming workload.
Given the previous description of autoscaling states, it can be concluded that there is a delay of (vUp+$inUp$) between the state of under-provisioned and the state of the autoscaling process taking an action in such a context. Based on this assumption, we propose the number of forecast data points should be equal to or greater than the delay of reaction of the autoscaling process as

$$m > (vUp+inUp)/ \mathrm{I}] \qquad (1)$$

where I is a single unit of observation intervals.

The upper threshold and the lower threshold *thrDown* are also considered as time series matrix. We estimate the upper threshold *thrUp* as the exponentially weighted moving average plus the deviation, i.e.

$$thrUp(t) = X(t) + \sigma \qquad (2)$$

Likewise, the lower threshold *thrDown* is estimated as the exponentially weighted moving average minus the deviation, i.e.

$$thrDown(t) = X(t) - \sigma \qquad (3)$$

B. Model Calibration

We use the Google trace data [4] as samples to demonstrate the calibration of the algorithm variables with time series data. The Google trace is of production workloads running on Google clusters collected over 6 hours and 15 minutes. The dataset contains over 3 million observations (or data points) with the following features: 1) Timestamp in seconds; 2) JobID as unique job identifier; 3) TaskID as unique task identifier; 4) JobType; 5) Normalized Task Cores as normalized number of CPU cores used; and 6) Normalized Task Memory as normalized value of the average memory consumed by the task.

The trace data provides sufficient samples for us to experiment with the workload analysis techniques we propose to fit with our autoscaling algorithm. In our experiments, we consider a slice of the trace data as 10,000 data points. Then the interval between the data point is equivalent to I, the single unit of observation variable in the algorithm presented in List

1. In this case, the variable Z, the time duration of one partition of the analysis is $Z = 10{,}000 \times I$ for a slice stripped from the trace. We further normalize the CPU and memory usage to 100% by means of linear scaling each data point according to the minimum and maximum values of the slice.

Figure 5 shows the predicted curve $X(t)$ using weighted moving average. The upper threshold curve *thrUp* and lower threshold curve *thrDown* are also calculated according to Eq. 4 and Eq. 5. The data points beyond the upper threshold curve and those below the lower threshold curve are outliers. By counting the number of outliers and their ratio to the total size of the data points in the analysis slice, we can make an approximation of the likelihood that the upper or lower threshold is across as P_u and P_d respectively.

$$P_u = \frac{Number\ of\ outliners\ beyond\ thrUp}{Total\ data\ size} \quad (4)$$

$$P_d = \frac{Number\ of\ outliners\ below\ thrDown}{Total\ data\ size} \quad (5)$$

The higher the likelihood, the more time the workload remains either under-provisioned or over-provisioned. Hence, P_u x Z and P_d x Z can be compared to the time-out variables *vUp* and *vDown* respectively to decide if an autoscaling action alert should be triggered. *inUp* and *inDown* represent the mandatory time to wait before triggering an alert once an alert has already been triggered. The values of these variables are set by measuring the average response time of the resource provision and de-provision actions in the autoscaling process. Note that the values of *inUp* and *inDown* may not necessarily be the same.

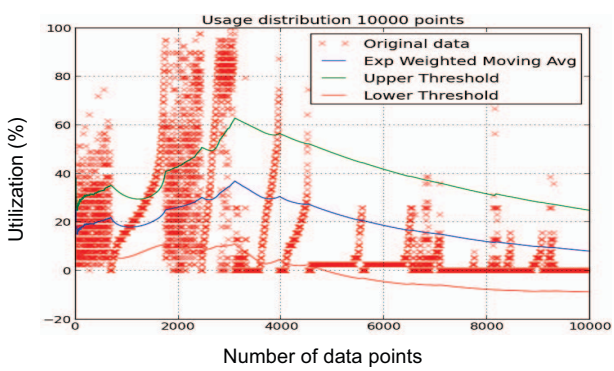

Figure 5 Workload prediction with weighted moving average

C. *End-to-End Delay Estimation*

The combination of the algorithm and time series data analysis allows the estimation of end-to-end responsiveness of the autoscaling workflow. We further represent the workflow in List 1 in a Petri Net model. We use a Petri Net workflow modeling tool, called WoPeD [10]. The notations in WoPeD are depicted in Figure 7 that include the states (in circle), transitions (in square) and resources (in a bar with an arrow). The transition notation in WoPeD can have branches to represent the fork-and-join activities. This kind of transition is represented by a square with an arrow. The resource notation in WoPeD can be associated with a transition to indicate the time attribute of the transition. For example, the transition "*checking status*" in Figure 7 models the interval of data points in time series, which represents the interval of collecting the monitoring data from the cloud architecture. This interval is equivalent to the single unit of observation variable I in our autoscaling algorithm. In this case, the resource "*timer*" is associated with the transition "*checking status*" and the time attribute is set to 30 seconds.

The model represents the workflow that in an interval of 30 seconds, the autoscaling process checks the status of a metrics of interest (such as workload of CPU or memory, or arriving rates, or queueing length etc.). The input of the model is the predicted workload as discussed in the above two sections. The model further proceeds to the "*analyzing*" state, which decides if the threshold value is across. The transition of "*threshold across*" has two branches. The number on each branch of the transition is the probability of the branch. If the threshold is not across, the autoscaling process transits to the "*stable*" state, and waits for the next circle of status checking with the interval of 30 seconds (as shown in the state "*next check*"). The probability of this branch is (1-P_d) or (1-P_u). If the threshold is across, for example, the autoscaling process transits to the state of "*under/over provisioned*". The probability of this branch is P_u or P_d. Then, the autoscaling process has two possibilities indicated by the branching transition "*observing*". If the under provisioned state lasts more than 5 minutes as the timer resource associated with the "*observing*" transit, the autoscaling process transits to the state of "*autosclaing triggered*". Otherwise, it transits to the "*stable*" state without any further autoscaling action and waits for the next circle of status checking. Following the state of "*autoscaling triggered*", the transition is "*autoscaling*" with the time attribute indicating the time required for an autoscaling action.

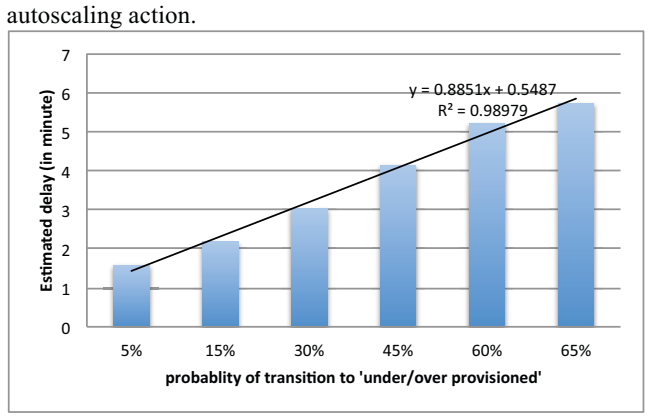

Figure 6 Estimated end-to-end delay of autoscaling workflow

Assume the rate of the "*observing*" branches is to "50%/50%", that means half of the time the workload is beyond or below the threshold for long enough to trigger the autoscaling actions. This is to avoid too frequent autoscaling actions, which could also degrade the overall performance. Following the workload prediction techniques discussed in section IV.A., the sample data of the size of 10K, 20K, 30K, 40K, and 50K of the Google trace yield the probability of P_u or P_d in the range of approximately 5% to 65%. The average

end-to-end delay of the autoscaling workflow is estimated by tuning the probability of the transition from the state "threshold across" from 5% to 65%. The result is depicted in Figure 6 that shows the estimated end-to-end delay has a linear trend as the size of outliers beyond or below the threshold value increases.

V. RELATED WORK

Lorido-Botran et al. [1] focused on the current issues of auto-scaling from the IaaS's client perspective. This survey provided a category of autoscaling techniques into five aspects, namely static, threshold-based polices, reinforcement learning, queueing theory, control theory and time-series analysis.

Mao et al. [12] described a provisioning method that automatically adjusts to workload changes. Their work is based on a monitor-control loop that adjusts to dynamic changes such as the workload bursting and delayed instance acquisitions. In **Error! Reference source not found.**, the resource provisioning problem was formulated in a two-phase algorithm. The first phase focused on providing optimal resources by proposing mathematical formulae. The second phase proposed a Kalman filter prediction model to predict demand. If above techniques produce workload prediction in time series, the prediction can be input to the autoscaling algorithm we propose in this paper.

A novel framework proposed in [13] supported reactive and proactive approaches for implementing autoscaling services. With a reactive approach, resources are scaled in or out in response to fluctuating user demands. With a proactive approach, on the other hand, future demand is predicted. Resources are scaled in advance for the increased or decreased demand. The authors developed a set of predictors for future demand on infrastructure resources, as well as a selection mechanism to choose the best predictor. The proactive approach was more successful at minimizing cost and SLO violations, while the reactive approach was useful for reducing resources that had already been over-provisioned.

VI. CONCLUSION

In this paper, we model the autoscaling operations in commonly used cloud service providers. The model explores time series data and workload prediction techniques to capture the dynamics of the autoscaling workflow. We use Google trace data as sample data to calibrate the model. In addition, the model is represented in another analytical model of Petri Net. We show the resulting Petri Net model to estimate the end-to-end delay of the autoscaling workflow given what-if analysis.

REFERENCES

[1] Tania Lorido-Botran, Jose Miguel-Alonso, and Jose A. Lozano. A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments. Journal of Grid Computing, pages 1–34, 2014.

[2] Nathan D. Mickulicz and Priya Narasimhan and Rajeev Gandhi, To Auto Scale or Not to Auto Scale, Proceedings of the 10th International Conference on Autonomic Computing (ICAC 13), 145—151, 2013.

[3] Qinghua Lu and Liming Zhu and Xiwei Xu and Len Bass and Shanshan Li and Weishan Zhang and Ning Wang , Mechanisms and Architectures for Tail-Tolerant System Operations in Cloud, 6th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 14), 2014.

[4] Hellerstein, J. L. Google Cluster Data. http://googleresearch.blogspot.com/2010/01/google-cluster-data.html.

[5] Amazon Web Services: //aws.amazon.com/documentation/autoscaling/.

[6] OpenStack: https://wiki.openstack.org/wiki/Heat.

[7] Windows Azure: http://msdn.microsoft.com/en-us/library/hh680945(v=pandp.50).aspx.

[8] Google Cloud Platform: https://cloud.google.com/developers/articles/auto-scaling-on-the-google-cloud-platform.

[9] Netflix: http://techblog.netflix.com/2013/11/scryer-netflixs-predictive-auto-scaling.html.

[10] WoPeD, Workflow Petri Net Designer http://woped.dhbw-karlsruhe.de/woped/

[11] H. Alipour, Y. Liu and A. Hamou-Lhadj, "Analyzing Auto-scaling Issues in Cloud Environments," To Appear in Proceedings of the IBM Center of Advanced Studies Conference (CASCON), 10 pages, 2014.

[12] M. Mao and M. Humphrey, Auto-scaling to minimize cost and meet application deadlines in cloud workflows, Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, p. 49, 2011.

[13] Ren-Hung Hwang; Chung-Nan Lee; Yi-Ru Chen; Da-Jing Zhang-Jian, "Cost Optimization of Elasticity Cloud Resource Subscription Policy," Services Computing, IEEE Transactions on , vol.7, no.4, pp.561,574, Oct.-Dec. 2014

[14] F. J. Almeida Morais, F. Vilar Brasileiro, R. Vigolvino Lopes, R. Araujo Santos, W. Satterfield, and L. Rosa, Autoflex: Service agnostic auto-scaling framework for iaas deployment models, Proceeding of 13th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), 2013, pp. 42–49, 2013.

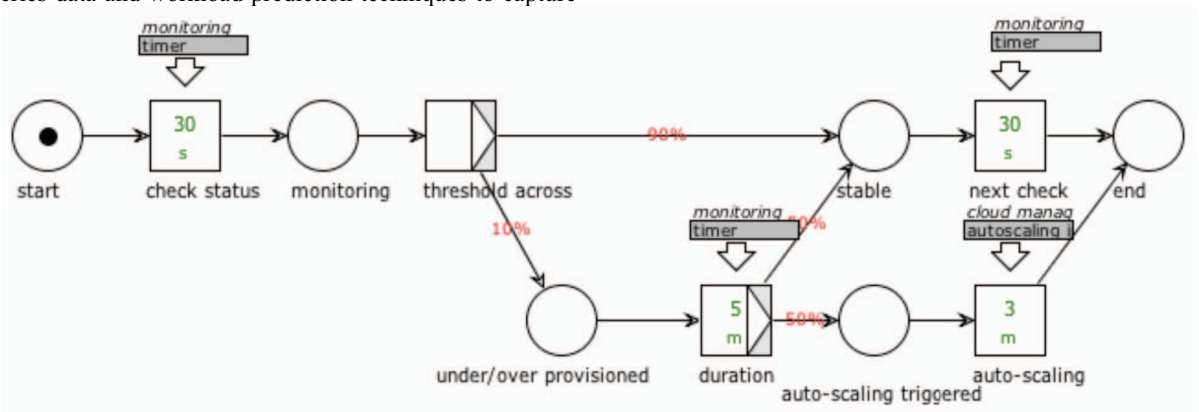

Figure 7 Petri Net model of the autoscaling workflow